Day 77: Positional Encoding, From Scratch — Giving Attention a Sense of Order

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 1 | Complete study guide

Watch: Positional embeddings in transformers EXPLAINED (AI Coffee Break with Letitia) | Positional Encoding in Transformer Neural Networks Explained (CodeEmporium)

Script: `day77_positional_encoding.py` · every number in this guide is copied by program from `day77_run_output.txt`, the real output of that script.

Self-attention cannot tell one word order from another (Day 74). The Transformer's fix is a vector per position, made of sines and cosines, added to the word vectors. Today it is built, its key property checked, and it is tested twice. On a toy task that needs position it works perfectly on the lengths it was trained on and mostly fails beyond them. On Week 9's real tagging task it helps only when it is strong in the attention scores and kept out of the word's own vector: then test accuracy is 0.8487, above Day 62's RNN (0.8327), with two heads that learned to read the next and the previous word.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **positional encoding** is a fixed vector for every position p, added to the vector of the word at that position. The sinusoidal one fills pairs of numbers with sin and cos of p times a different speed per pair: fast for the first pairs, very slow for the last. Two words with the same vector now give different inputs when they sit in different places.

REAL-WORLD SCENARIO

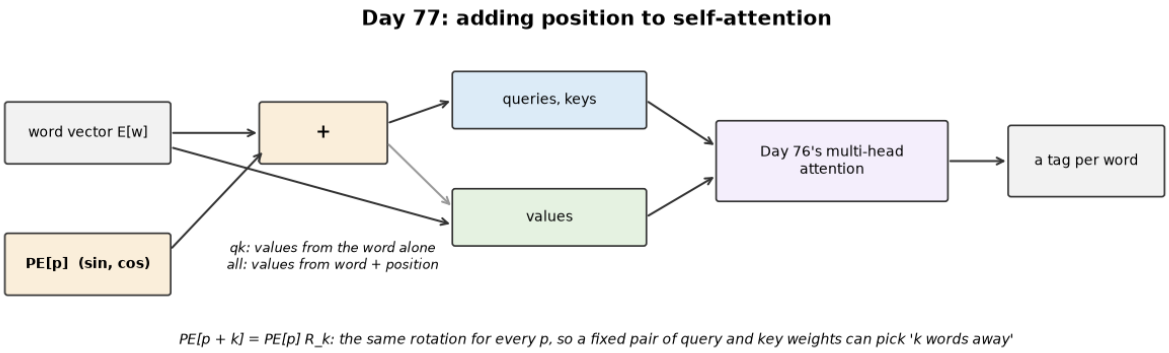
Seats in a theatre: the ticket says who you are, the seat number says where you are. A clock with an hour hand, a minute hand and a second hand tells any moment of the day by the angles of hands that turn at different speeds.

MATHEMATICS

$$PE[p, 2i] = \sin(p / 10000^{2i/d}), \quad PE[p, 2i+1] = \cos(p / 10000^{2i/d})$$

$$\text{input at position } p = E[\text{word}] + PE[p]$$

SYSTEM DIAGRAM



Where the encoding enters the attention layer: 'all' adds it to everything, 'qk' only to the queries and keys.

Week 12 is built part by part

Days 77–82 each add a short exercise script, one part of a Transformer; Day 83 trains a tiny Transformer built from them. Today's part is the encoding table (`day77_exercise.py`).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The encoding

DEFINITION — start here, no prior knowledge assumed

The table for d = 32: the first numbers at the first positions, and how many positions each pair needs to turn once around its circle.

REAL-WORLD SCENARIO

Reading the hands of a clock at 12:00, 12:01, 12:02: the second hand moves a lot, the hour hand barely.

Why this test

Seeing the numbers makes the 'many speeds' idea concrete.

IMPLEMENTATION

```
def sinusoidal_encoding(n, d):
    p = np.arange(n)[:, None]
    i = np.arange(d // 2)[None, :]
    angle = p / 10000 ** (2 * i / d)
    PE = np.zeros((n, d))
    PE[:, 0::2] = np.sin(angle)
    PE[:, 1::2] = np.cos(angle)
    return PE
```

ACTUAL OUTPUT

```
 position    col 0    col 1    col 2    col 3    col 4    col 5
        0   0.0000   1.0000   0.0000   1.0000   0.0000   1.0000
        1   0.8415   0.5403   0.5332   0.8460   0.3110   0.9504
        2   0.9093  -0.4161   0.9021   0.4315   0.5911   0.8066
        3   0.1411  -0.9900   0.9933  -0.1160   0.8126   0.5828
        4  -0.7568  -0.6536   0.7785  -0.6277   0.9536   0.3011
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
positions per full turn, pair by pair (pair 0 is columns 0-1):
  0: 6.3  1: 11.2  2: 19.9  3: 35.3  4: 62.8  5: 111.7  ...  15: 35333
Actual output from this lesson's run.
```

What it means

The first pair turns once every 6.3 positions, the last once every 35333: fast pairs tell neighbours apart, slow pairs tell the start of a long text from its end. Position 0 is (0, 1) in every pair.

WHY CHOOSE THIS?

No weights to learn; defined for any position.

WHY NOT THIS? (TRADEOFFS)

A fixed choice: nothing adapts it to the task.

2. Moving along is a rotation

DEFINITION — start here, no prior knowledge assumed

For each pair, going k positions further turns the point (sin, cos) by the same angle, wherever it starts. So one matrix R_k, built from k alone, maps every PE[p] to PE[p + k].

REAL-WORLD SCENARIO

Ten minutes later the minute hand has turned the same 60 degrees, whatever the time was.

MATHEMATICS

$$\sin(a + b) = \sin a \cos b + \cos a \sin b, \quad \cos(a + b) = \cos a \cos b - \sin a \sin b$$

$$\Rightarrow PE[p + k] = PE[p]\, R_k \quad \Rightarrow \quad PE[p] \cdot PE[p + k] = \Sigma_i \cos(k\, \omega_i), \text{ the same for every } p$$

SPECIFICATION

Check	k = 1, 2, 5, 20; every p = 0..99; d = 32.

Why this test

This property is the reason for sines and cosines: a fixed linear map can turn 'here' into 'k words away'.

IMPLEMENTATION

```
def shift_rotation(k, d):
    R = np.zeros((d, d))
    for i in range(d // 2):
        a = k / 10000 ** (2 * i / d)
        R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]]
    return R
```

ACTUAL OUTPUT

```
   k   largest |PE[p+k] - PE[p] R_k|   PE[p] . PE[p+k]: smallest   largest
   1                       3.71e-15                     15.3136   15.3136
   2                       6.27e-15                     13.7301   13.7301
   5                       4.39e-15                     11.7774   11.7774
  20                       3.57e-15                     10.4709   10.4709
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
PE[0] . PE[k] for k = 0..8 (k = 0 gives d / 2 = 16; nearby positions score highest, but not in a
    smooth line):
  16.00  15.31  13.73  12.27  11.66  11.78  11.89  11.43  10.52
Actual output from this lesson's run.
```

What it means

The rotation is exact to rounding (3.71e-15 at k = 1) and the dot product is the same at every p (15.3136 at k = 1). It is largest at k = 0 (16.00) and lower further away, but not in a smooth line (11.66, 11.78, 11.89 at k = 4, 5, 6).

WHY CHOOSE THIS?

Relative offsets are built in.

WHY NOT THIS? (TRADEOFFS)

Only for the encoding itself; once mixed with word vectors and weights, nothing forces the model to use offsets (Sections 4 and 6).

3. Order becomes visible

DEFINITION — start here, no prior knowledge assumed

Day 74's shuffle test on Day 76's multi-head layer: shuffle the words, keep the positions, and see whether each word's output simply moves with it.

REAL-WORLD SCENARIO

Swapping two people's seats changes what each of them can see from where they sit.

ACTUAL OUTPUT

```
shuffle [4, 5, 3, 1, 2, 6, 0]
no encoding        : largest |output(shuffled) - shuffled(output)| = 8.88e-15
with the encoding  : largest |output(shuffled) - shuffled(output)| = 3.18e+01
Actual output from this lesson's run.
```

What it means

Without the encoding the outputs just move with the words (8.88e-15, rounding); with it they change (3.18e+01 with these random weights). The layer can now see order; whether a trained model uses it is the question of the next sections.

WHY CHOOSE THIS?

One addition removes the order blindness.

WHY NOT THIS? (TRADEOFFS)

Being able to see order is not the same as using it.

4a. Gradient check

DEFINITION — start here, no prior knowledge assumed

The toy network with no encoding, the sinusoidal one and a learned table, and the tagger with the encoding nowhere, everywhere ('all') or only in queries and keys ('qk').

REAL-WORLD SCENARIO

Testing each variant's wiring before comparing them.

MATHEMATICS

qk: Q, K from E[w] + PE; V and the word's own vector from E[w] alone

dE[w] collects dQ W_q^T *+ dK* W_k^T *+ dV* W_v^T *+ the word-vector path; the sinusoidal PE has no gradient*

IMPLEMENTATION

```
def positional_attention(Xqk, Xv, P, h, mask):
    Q, K = split_heads(Xqk @ P["Wq"], h), split_heads(Xqk @ P["Wk"], h)
    V = split_heads(Xv @ P["Wv"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P["Wo"], dict(Q=Q, K=K, V=V, A=A, Z=Z)
```

ACTUAL OUTPUT

```
toy, none       : largest |numeric - analytic| = 4.93e-11; every entry passes: True
toy, sin        : largest |numeric - analytic| = 5.41e-11; every entry passes: True
toy, learned    : largest |numeric - analytic| = 5.10e-11; every entry passes: True
tagger, none    : largest |numeric - analytic| = 7.42e-10; every entry passes: True
tagger, all     : largest |numeric - analytic| = 7.90e-10; every entry passes: True
tagger, qk      : largest |numeric - analytic| = 7.54e-10; every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes for all six models.

WHY CHOOSE THIS?

The comparisons below rest on checked gradients.

WHY NOT THIS? (TRADEOFFS)

Tiny models; the trained ones run the same code.

4. A task that needs position: write the previous token

DEFINITION — start here, no prior knowledge assumed

Random tokens 1–9; at every position the answer is the token just before it. Without position, a position cannot even tell which token is its neighbour.

REAL-WORLD SCENARIO

Repeating what the person before you in a queue said: you need to know who stood before you.

SPECIFICATION

Models	One head, size 32: no encoding, sinusoidal, or a learned table (one free vector per position).
Training	Sequences of 8–16 tokens; SGD lr 0.1, clip 1.0, batches of 32, 3,000 steps, three seeds.
Test	500 fixed sequences each of 16 tokens, and of 24, 32 and 48: longer than any training sequence.

Why this test

It needs position and nothing else, and the longer tests ask whether the encoding works on positions never seen.

IMPLEMENTATION

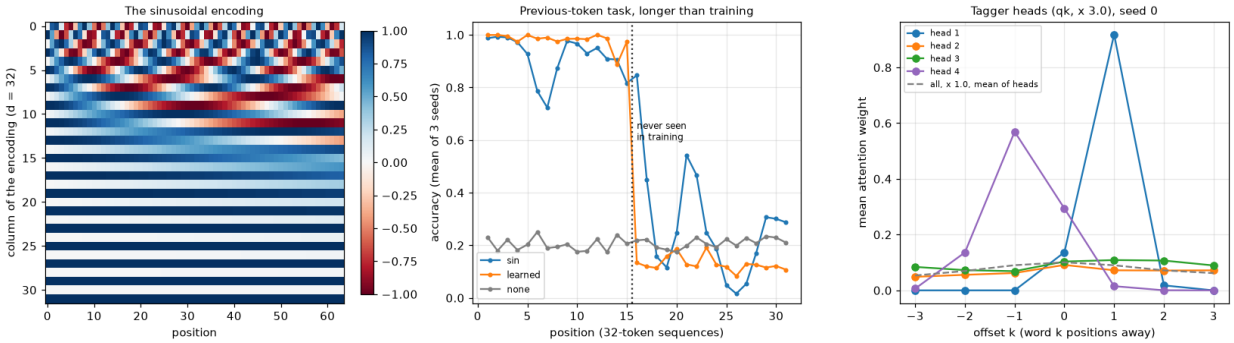
```
def forward(self, tok):
    P = self.P
    n = tok.shape[1]
    X = P["E"][tok]
    if self.pe == "sin":
        X = X + self.SIN[:n]
    if self.pe == "learned":
        X = X + P["PE"][:n]
    Q, K, V = X @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
    Z, A = scaled_dot_product_attention(Q, K, V)
    p = softmax(Z @ P["Wout"] + P["bout"])
    self.cache = dict(tok=tok, X=X, Q=Q, K=K, V=V, A=A, Z=Z, p=p)
    return p
```

ACTUAL OUTPUT

```
encoding       n=16    n=24    n=32    n=48   n=16 per seed
none          0.253   0.227   0.206   0.182   0.251 0.254 0.254
sin           1.000   0.775   0.585   0.374   1.000 1.000 0.999
learned       1.000   0.695   0.542   0.383   1.000 1.000 1.000
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
at n = 32, by position (mean of seeds): positions 1-15 were seen in training, 16-31 never
  sin      positions 1-15: 0.914   positions 16-31: 0.278
  learned  positions 1-15: 0.981   positions 16-31: 0.130
Actual output from this lesson's run.
```



Left: the encoding. Middle: the previous-token task at length 32, by position (positions after 15 were never seen in training). Right: what the tagger's four heads look at (Section 6).

What it means

Without position the model reaches only 0.253; above the 0.111 of guessing, probably because it can at least guess a token that occurs in the sequence (not tested here). **With either encoding** it is perfect on 16 tokens (1.000).

Neither works much beyond training: at 32 tokens the sinusoidal model gets 0.278 of the never-seen positions right and the learned table 0.130. The learned table has never trained those rows, so it has nothing there; the sinusoidal encoding is defined there, but the trained weights were fitted only to the positions they saw, and even the seen positions drop to 0.914 in longer sequences. 'Defined for any length' does not mean 'works at any length'.

WHY CHOOSE THIS?

Shows exactly what the encoding gives: order within the trained range.

WHY NOT THIS? (TRADEOFFS)

A toy task, one head; longer training ranges or other encodings may extrapolate better.

5. Real tagging: where to add the encoding, and how strongly

DEFINITION — start here, no prior knowledge assumed

Day 62's data, Day 74's tagger with Day 76's 4-head layer. 'all' adds the encoding to the word vectors, as in the Transformer paper; 'qk' adds it only to the inputs of the queries and keys, where the layer decides where to look. Placement and strength are chosen on validation (seed 0); then three seeds on the test set.

REAL-WORLD SCENARIO

Writing a seat number on a guest's name badge in large print: it helps the usher find seats, but if it covers the name, nobody knows who the guest is.

SPECIFICATION

Training	SGD lr 2.0 (Day 74's), clip 1.0, batches of 32, 25 epochs, best epoch on validation.
Grid	strength × 0.3, 1, 3, 10 for 'all' and 'qk'; word vectors start at spread 0.1.

Why this test

The real question: does knowing position help a self-attention tagger, and Day 68's RNNs, which see order by construction, set the bar.

IMPLEMENTATION

```
def forward(self, ids):
    P = self.P
    m, T = ids.shape
    self.ids, mask = ids, ids != PAD
    X = P["E"][ids]
    pos = self.PE[:T][None] if self.place != "none" else 0.0
    self.Xqk = X + pos
    self.Xv = X + pos if self.place == "all" else X
    kmask = np.repeat(mask[:, None, None, :], self.h, axis=1).reshape(m * self.h, 1, T)
    self.O, self.c = positional_attention(self.Xqk, self.Xv, P, self.h, kmask)
    self.XZ = np.concatenate([self.Xv, self.O], axis=2).reshape(m * T, -1)
    self.Hh = np.tanh(self.XZ @ P["W1"] + P["b1"])
    self.probs = softmax(self.Hh @ P["Why"] + P["by"])
    return self.probs
```

ACTUAL OUTPUT

```
place      x 0.3    x 1.0    x 3.0   x 10.0
all       0.7980   0.7989   0.1400   0.1400
qk        0.8009   0.8265   0.8381   0.7876
no encoding: 0.8001
chosen on validation: all x 1.0, qk x 3.0
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
model                        seed 0   seed 1   seed 2     mean
no encoding                  0.8042   0.8049   0.8018   0.8036
all, x 1.0                   0.8105   0.8023   0.8062   0.8063
qk, x 3.0                    0.8485   0.8480   0.8496   0.8487
for comparison: Day 74's single-head tagger 0.8070, Day 62's RNN 0.8327, Day 68's LSTM 0.8377,
    bidirectional LSTM 0.8577
Actual output from this lesson's run.
```

What it means

'all' barely helps (0.8063 against 0.8036 with no encoding), and at strength 3 or 10 training collapses (0.1400: one tag for every word). A likely reason: at strength 3 the encoding's numbers reach 3, thirty times the word vectors' starting spread of 0.1, so the word itself is drowned.

'qk' at strength 3 works: 0.8487 over three seeds, above Day 62's RNN (0.8327) and Day 68's LSTM (0.8377), below the bidirectional LSTM (0.8577). Keeping the encoding out of the values and the word's own vector lets it be strong where it matters without drowning the word.

What was checked before believing it. During development, both placements were also tried at matched small strengths (0.1, 0.3): neither used position then. So it is not 'qk' by itself that matters but a strong position signal in the scores, which only 'qk' allows here. Those development runs are not part of this script's output.

WHY CHOOSE THIS?

A real gain from one addition, chosen honestly on validation.

WHY NOT THIS? (TRADEOFFS)

The best strength depends on the word vectors' scale; a model with layer normalisation (Day 78) may behave differently.

6. What the heads look at

DEFINITION — start here, no prior knowledge assumed

The mean weight each head puts on the word k positions away, over 500 test sentences.

REAL-WORLD SCENARIO

Watching where each usher looks: one at the seat to the left, one to the right.

Why this test

Section 5 shows that accuracy rose; this shows how.

ACTUAL OUTPUT

```
no encoding:       k=-3    k=-2    k=-1     k=0     k=1     k=2     k=3
  head 1           0.056   0.061   0.072   0.086   0.074   0.062   0.056
  head 2           0.056   0.062   0.074   0.079   0.074   0.063   0.056
  head 3           0.057   0.062   0.074   0.078   0.074   0.064   0.056
  head 4           0.055   0.061   0.072   0.088   0.073   0.062   0.056
all, x 1.0:       k=-3    k=-2    k=-1     k=0     k=1     k=2     k=3
  head 1           0.050   0.065   0.092   0.114   0.113   0.089   0.069
  head 2           0.039   0.050   0.063   0.076   0.077   0.072   0.069
  head 3           0.053   0.059   0.072   0.067   0.079   0.067   0.062
  head 4           0.072   0.105   0.134   0.146   0.091   0.059   0.046
qk, x 3.0:       k=-3    k=-2    k=-1     k=0     k=1     k=2     k=3
  head 1           0.000   0.000   0.000   0.136   0.918   0.018   0.000
  head 2           0.049   0.056   0.062   0.091   0.072   0.071   0.072
  head 3           0.084   0.073   0.069   0.103   0.109   0.107   0.089
  head 4           0.006   0.136   0.569   0.294   0.015   0.000   0.000
Actual output from this lesson's run.
```

What it means

With the 'qk' encoding, head 1 puts 0.918 of its weight on the **next** word and head 4 0.569 on the **previous** one: the two neighbours Day 68's probe found a bidirectional tagger relies on. Without an encoding, or with 'all', every head is much flatter (no weight above 0.146). This is Day 76's lesson again: two things to fetch, two heads.

WHY CHOOSE THIS?

The mechanism is visible, not guessed.

WHY NOT THIS? (TRADEOFFS)

Seed 0 only.

7. Day 62's shuffle test on the real test set

DEFINITION — start here, no prior knowledge assumed

Every test sentence with its words shuffled, each word keeping its true tag.

REAL-WORLD SCENARIO

Reading a sentence with its words in random order: if you still understand it, you were not using the order.

ACTUAL OUTPUT

```
model                       normal order  shuffled
no encoding                       0.8042    0.8042
all, x 1.0                        0.8105    0.8000
qk, x 3.0                         0.8485    0.7376
Actual output from this lesson's run.
```

What it means

Without an encoding nothing changes (0.8042 both ways). With 'qk' accuracy drops from 0.8485 to 0.7376: the tagger now relies on word order, as Day 62's RNN did (0.7487 shuffled).

WHY CHOOSE THIS?

Proves the model uses order.

WHY NOT THIS? (TRADEOFFS)

One shuffle per sentence, seed 0.

Interview preparation — technical questions

Q: Write the sinusoidal encoding.

A: PE[p, 2i] = sin(p / 10000^(2i/d)), PE[p, 2i+1] = cos(p / 10000^(2i/d)).

Q: Why sines and cosines?

A: PE[p + k] = PE[p] R_k for every p: shifting is one fixed rotation, so PE[p] . PE[p+k] depends only on k.

Q: How fast do the pairs turn here (d = 32)?

A: From once every 6.3 positions to once every 35333.

Q: What did the shuffle test give with and without it?

A: 8.88e-15 without (order-blind); 3.18e+01 with.

Q: Sinusoidal or learned, beyond the training length?

A: Both mostly failed here: 0.278 and 0.130 on never-seen positions.

Interview preparation — theoretical questions

Q: Why does self-attention need a positional encoding at all?

A: Its output does not change when the words are shuffled (Day 74).

Q: Is adding it to the word vector the only option?

A: No: today's best tagger added it only to queries and keys; rotary encodings (RoPE) also act on queries and keys only.

Q: Why did 'all' at strength 3 fail?

A: Probably because its numbers reached 30 times the word vectors' scale; training collapsed to one tag (0.1400).

Q: Does a defined-everywhere encoding extrapolate?

A: Not automatically: the weights were fitted to the positions seen in training.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A sinusoidal positional encoding gives self-attention a sense of order, and moving along by k positions is the same rotation everywhere. On real tagging it paid off only when it was strong in the attention scores without drowning the word: then two heads learned to read the next and the previous word, and accuracy rose from 0.8036 to 0.8487. It did not make the model work on lengths it had never seen.

- PE[p + k] = PE[p] R_k to 3.71e-15; PE[p] . PE[p + k] is the same for every p.
- Previous-token task: 1.000 within the trained lengths; 0.278 (sinusoidal) and 0.130 (learned) beyond them.
- Tagging: no encoding 0.8036, 'all' 0.8063, 'qk' x 3 0.8487 (Day 62's RNN 0.8327).
- Heads: 0.918 on the next word, 0.569 on the previous one.
- Shuffled test set: 0.8485 to 0.7376 with the encoding; no change without it.

Suggested watch order, if starting from zero

- 1. Positional Encoding in Transformer Neural Networks Explained (CodeEmporium) — the formula, step by step.
- 2. Positional embeddings in transformers EXPLAINED (AI Coffee Break with Letitia) — why sines, and the alternatives (learned, relative).

Day 78 preview

- **The Transformer encoder block.** Attention, a residual connection, layer normalisation and a position-wise feed-forward network, built from scratch, stacked, and tested on the same tagging task.
- Today's exercise script, `day77_exercise.py`, is the encoding table as one function, with its rotation and order checks: the first part of Day 83's Transformer.